

FAKE label = 0

REAL label = 1

Fake vs Real News Classification

An end to end text classification pipeline: from raw headlines to best performing models

Team: Martin, Jenifar, Yoan, Manu

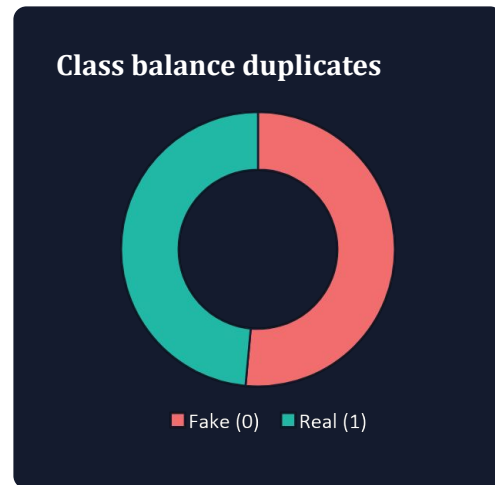
The problem and the data

1,946

exact duplicate rows detected in the raw data

Within the group of headlines `y_prediction_probability` below 0.8 (1155 headlines) there are 25% duplicates.

(7% of all headlines are duplicates.)



Roughly balanced: 51% fake, 49% real

Removing duplicates first

The raw file held 1,946 exact duplicate headlines. Dropping them before the split keeps the same text out of both train and test.

Before

34,152

rows, with duplicates

−1,946

After

32,206

unique headlines

Why it matters

A duplicate in both splits lets the model memorise a row it will later be tested on, inflating the score.

Effect on results

Every accuracy in this deck is measured on the de-duplicated data. Scores sit a little lower than a run with duplicates left in, but they are a more honest estimate of how the model handles genuinely unseen headlines.

Cleaning: diagnose first, then act

A diagnostic pass counted quality issues per class before any text was changed, so cleaning choices were evidence based.



Special characters

Present in both classes at similar rates

Decision: Removed — not a class signal



Very short texts

9 found, only in fake news

Decision: Kept — a genuine fake-news cue



Extra whitespace

192 found, only in real news

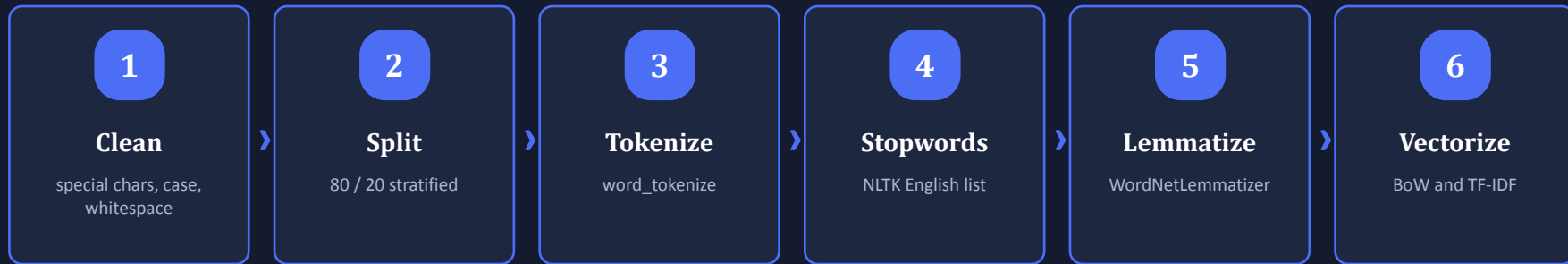
Decision: Normalised during cleaning

Cleaning applied to the whole set

remove special characters → normalise whitespace → strip → lowercase

From raw text to features

Split before anything is fit. The vectorizer learns only from training text, then transforms the test set.



Train / test split

27,321 training **6,831** test

Five models, one feature philosophy

Four classical models on sparse TF-IDF features, plus one dense embedding model and one pretrained transformer for contrast.

1

Logistic Regression

Default TF-IDF, default classifier — the baseline

2

Logistic Regression, tuned

C = 3, TF-IDF with max_df 0.8, min_df 2, bigrams

3

Multinomial Naive Bayes

Same tuned TF-IDF, probabilistic classifier

4

LightGBM + Word2Vec

Dense 100-d document vectors, gradient boosting

5

Pretrained transformer

Off-the-shelf sentiment model, used as a probe

Logistic Regression face-off

Model 1 uses default TF-IDF. Model 2 adds fine-tuning: max_df 0.8, min_df 2, n-grams=(1,2) and C = 3. The accuracy gain is marginal.

Model 1 · default

Plain TF-IDF, default LogReg

92.52% test

Train 95.44% · gap 2.92%

Model 2 · fine-tuned

max_df 0.8, min_df 2, n-grams=(1,2), C = 3

92.63% test **+0.11%**

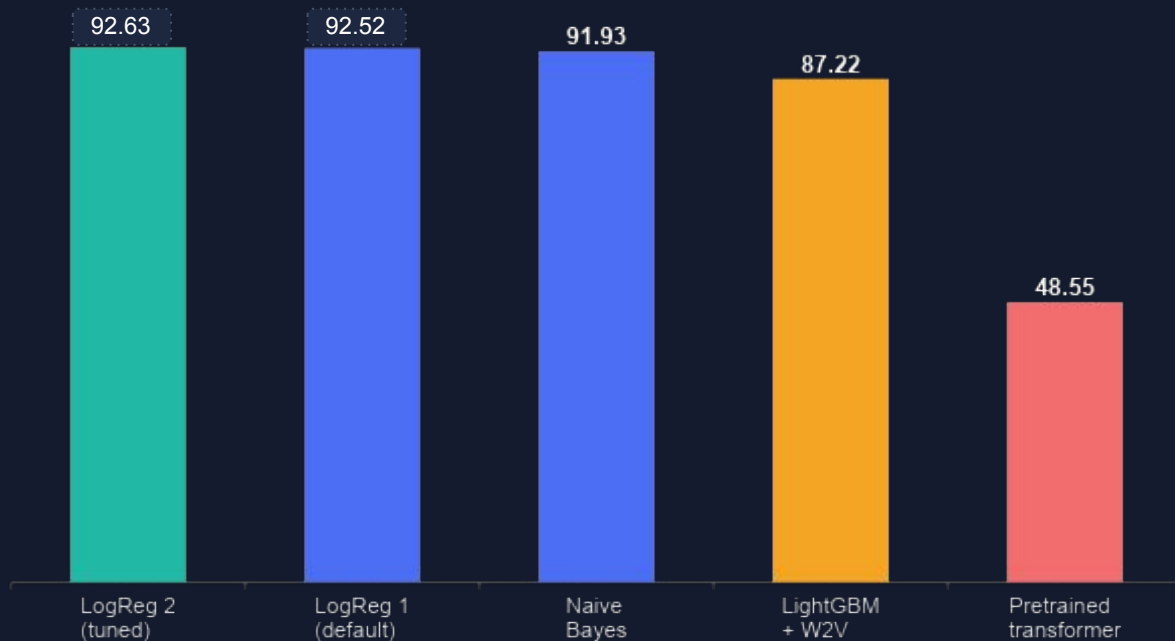
Train 95.96% · gap 3.33%

What the fine-tuning bought

Vocabulary trimming and bigrams lifted test accuracy by only 0.11 points. In short, keyword-driven headlines the default TF-IDF already captures most of the signal, so heavier feature engineering gives diminishing returns. Model 2 was still chosen for deployment for its cleaner, capped feature space.

Best logistic regression vs the field

Held-out 6,442 headline test split. The tuned logistic regression leads; the pretrained transformer collapses to near chance.



92.63%

Best & deployed · tuned LogReg

1.52%

Smallest train-test gap · Naive Bayes

48.55%

Pretrained transformer · wrong tool

A financial-sentiment model has no notion of fake vs real, so it predicts one class and stalls near 49%.

Accuracy

Train vs test exposes overfitting. The grid search reached a higher test score but opened the widest gap, so it was set aside.



Train - test gap

2.92%

LogReg
default

4.12%

GridSearch
C=10

3.33%

LogReg Tuned
C=3

1.52%

Naive
Bayes

3.88%

LightGBM
+ W2V

Stable
generalization

Set aside

GridSearch C=3

93.01% test, but the widest 4.12% train-test gap.
Higher score, weaker generalisation.

Deployed

LogReg tuned (C=3)

92.63% test with a controlled 3.33% gap and a capped 5k feature space. Best balance of accuracy and stability.

Preprocessing - Raw Data

```
# Removing punctuations
```

```
X = df.text
```

```
X.replace("[^a-zA-Z]", " ", regex=True, inplace=True)
```

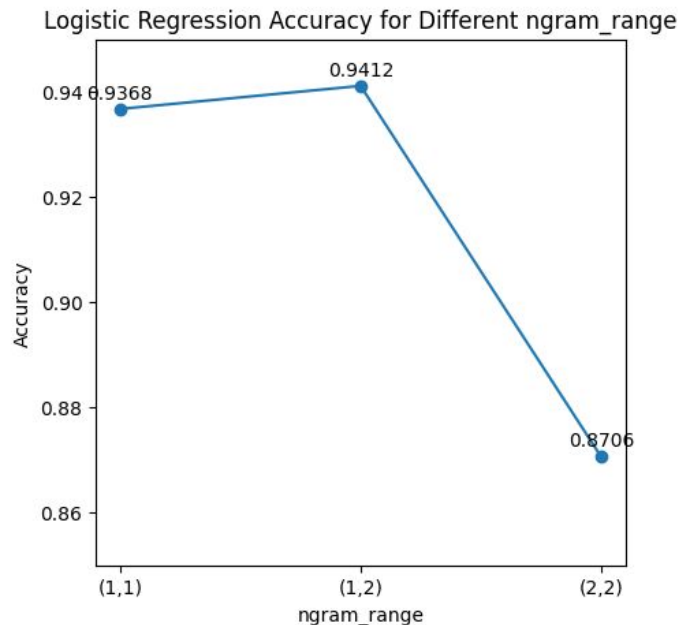
```
## implement BAG OF WORDS
```

```
countvector=CountVectorizer(ngram_range=(1,2), stop_words="english")
```

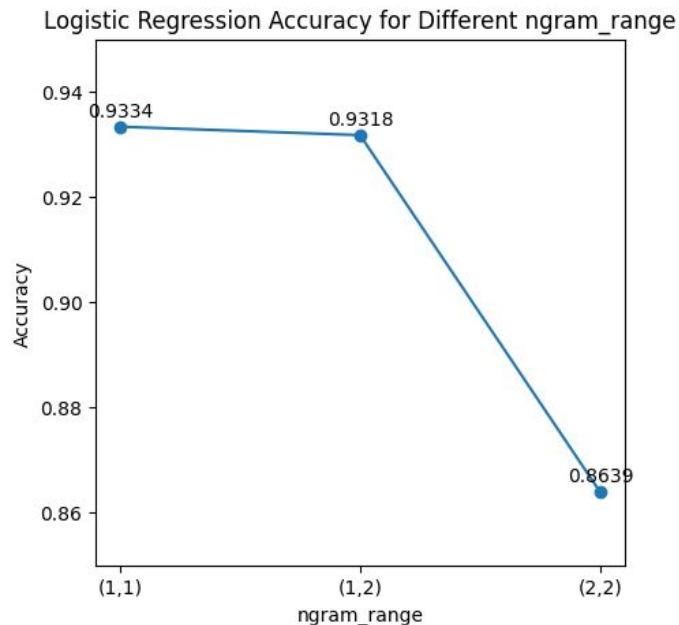
ngram_range tuning

Testing unigram, bigram and combined features

CountVectorizer



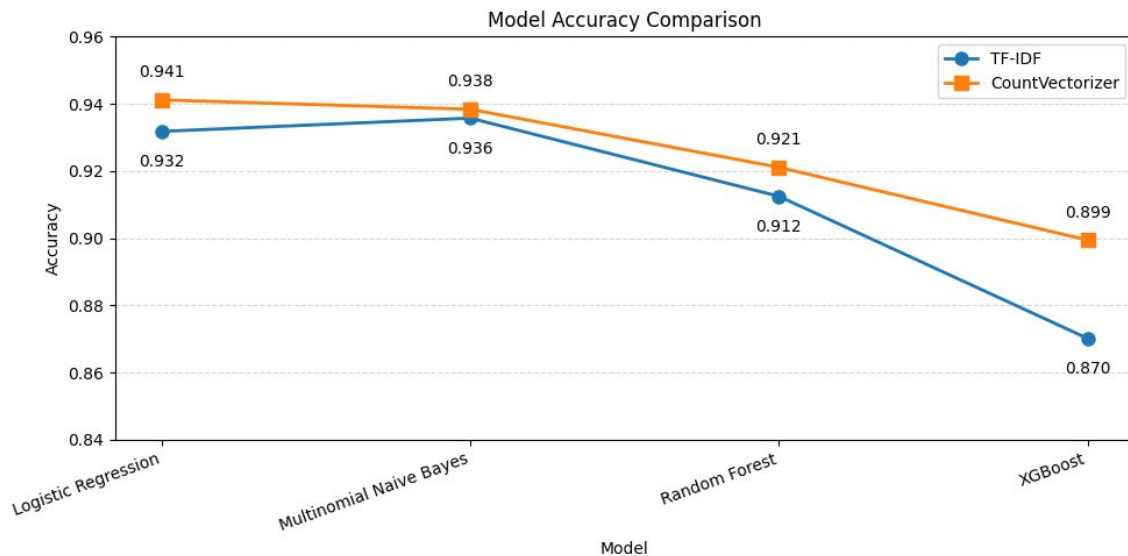
TF-IDF



Combined unigrams + bigrams, ngram_range=(1,2), gives the best Logistic Regression result. Pure bigrams, ngram_range=(2,2), lose too much single-word signal.

Model comparison

Four classifiers compared with two vectorizers



What the chart shows

CountVectorizer is slightly stronger than TF-IDF for all tested models. Logistic Regression and Multinomial Naive Bayes are the best performers, while XGBoost performs lowest in this setup.

Best model

**Logistic Regression +
CountVectorizer**

Accuracy: 0.941

Comparing Two Pipelines

Two teammates, Jenifar and Yoan, built independent pipelines on the same training and test data — and both independently arrived at the same first fix: removing duplicates.

32,206 unique headlines left after both pipelines removed 1,946 duplicate rows

5 models compared by Jenifar — including embeddings and a transformer

8 vectorizer × model combinations retested by Yoan on the deduplicated set

What each pipeline tested

Jenifar

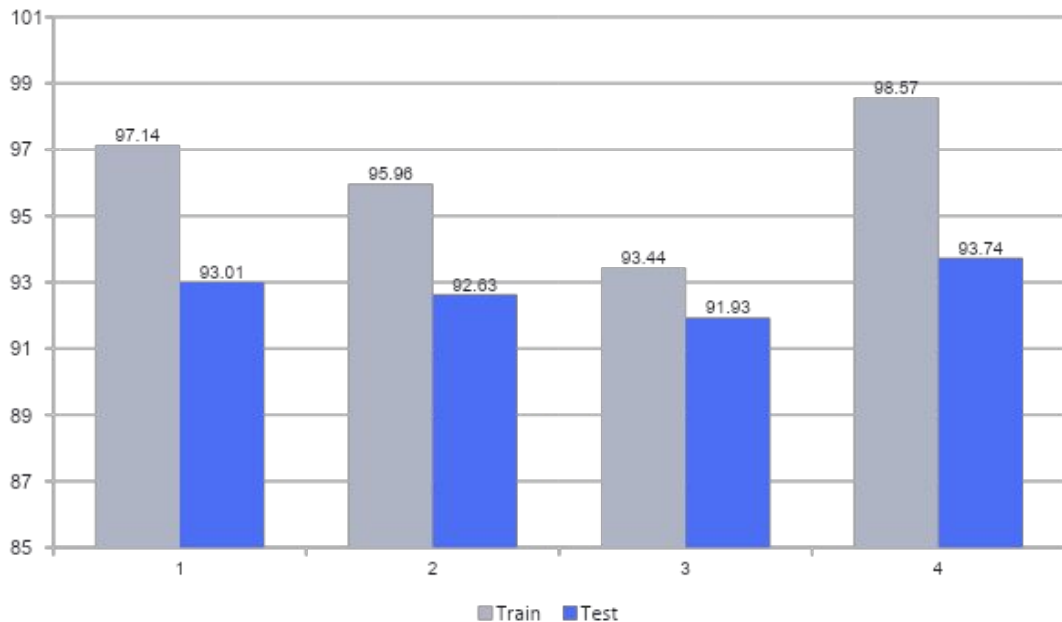
Full NLTK pipeline (stopwords + lemmatization), TF-IDF, 5 models — including Word2Vec + LightGBM and a pretrained transformer

Yoan

Lightweight cleaning, CountVectorizer / TF-IDF × 4 models, n-gram fixed at (1,2) after an earlier sweep

Accuracy Is Not the Full Picture

The two highest-scoring models across both notebooks also opened the widest train–test gaps.



High gap

GridSearch LR

4.12 pt gap

highest gap

NB + CountVec (Yoan)

4.83 pt gap — highest of all

BEST BALANCED

LR tuned (C=3)

3.33 pt gap

BEST GENERALIZATION

Naive Bayes (Jenifar)

1.52 pt gap — smallest of all

Three Winners, Three Criteria

Instead of declaring one best model, we ranked the candidates by what actually matters for each use case.



BEST RAW ACCURACY

93.74%

Yoan — Naive Bayes + CountVectorizer (1,2)

Disadvantage

Highest overfitting gap of the compared models (4.83 pts) — despite Naive Bayes usually being the ‘safe’ choice.



BEST GENERALIZATION

91.93%

Jenifar — Naive Bayes (capped TF-IDF)

Disadvantage

Lowest raw accuracy of the four; assumes word independence, a known simplification.



BEST BALANCE

92.63%

Jenifar — Tuned Logistic Regression (C = 3)

Disadvantage

Not #1 on either single metric — but the one the team actually shipped.

Bigger Isn't Always Better

Two more advanced approaches were tested. Neither outperformed the simple TF-IDF baseline.

Word2Vec + LightGBM

Every headline became one 100-d vector by averaging its word vectors, then LightGBM classified on top.

87.22%

test accuracy — below every TF-IDF model

Why lower? Averaging blurs word-level signal — for short headlines, sparse TF-IDF keyword evidence is simply stronger.

Pretrained Transformer

An off-the-shelf financial-sentiment model was applied directly, with no fine-tuning on this dataset.

48.55%

accuracy — essentially one-class guessing

Sentiment (positive/negative) doesn't map to veracity (fake/real) — it would need fine-tuning on this exact task to be a fair comparison.

What This Comparison Shows

1

No single winner

The best model changes depending on whether you optimize for raw accuracy or for generalization.

2

Preprocessing shapes overfitting too

The same algorithm, Naive Bayes, swung from a 1.5-point to a 4.8-point train–test gap purely on vocabulary capping and lemmatization — not just the model choice.

3

Match the tool to the task

A pretrained model only transfers when its training objective aligns with yours — sentiment $\neq$ veracity.

4

The Health Tech parallel

A model that memorizes training data but fails to generalize is a serious risk in production — whether it's flagging fake news or classifying clinical notes.

Generalization first: in production, reliability on unseen data matters more than a fraction of a point in accuracy — and that reliability comes from the full pipeline, not just the algorithm.